

Informe de Causa Raíz

1. Análisis del BUG-003: Falla en los datos del reporte PDF

- **Problema presentado:** Al generar un reporte en archivo PDF, las métricas actuales del dashboard del administrador se reflejan en ceros.
- **Técnica de los 5 ¿Por qué?:**
 1. ¿Por qué el PDF muestra ceros? Porque el documento se compila sin la información real de la base de datos.
 2. ¿Por qué no tiene la información real? Porque el componente de React enviaba un estado vacío al backend al momento de solicitar el PDF.
 3. ¿Por qué enviaba un estado vacío? Porque el botón de descarga se ejecutaba antes de que el hook de React terminara de actualizar las métricas en la interfaz.
 4. ¿Por qué el backend depende del frontend para los datos? Porque el endpoint en NestJS estaba diseñado para recibir los datos en el body en lugar de consultarlos por sí mismo.
 5. ¿Por qué se diseñó así? Por un fallo de arquitectura donde no se separó adecuadamente la responsabilidad de consulta.
- **Causa raíz:** Defecto en el diseño del endpoint en NestJS, el cual dependía de los estados de la interfaz de usuario en lugar de realizar una consulta directa y segura a la base de datos mediante Prisma ORM.
- **Consecuencia:** Pérdida de funcionalidad administrativa clave y reportes inutilizables.
- **Acción propuesta:** Refactorizar el endpoint en NestJS para que ejecute sus propias consultas a MySQL a través de Prisma antes de compilar el PDF.

2. Análisis del BUG-004: Falla al actualizar etiquetas del barbero

- **Problema presentado:** El sistema no guarda ni actualiza las etiquetas agregadas (ej. "Degradado", "Cejas") en el portafolio del barbero.
- **Técnica de los 5 ¿Por qué?:**
 1. ¿Por qué no se guardan las etiquetas? Porque el backend rechaza la actualización al interactuar con la base de datos.
 2. ¿Por qué la rechaza? Porque existe una incompatibilidad en el formato de los datos enviados.
 3. ¿Por qué hay incompatibilidad? Porque React envía las etiquetas como un Array, pero la base de datos espera una cadena de texto (String).
 4. ¿Por qué la base de datos espera un String? Porque en el esquema de Prisma, el campo de etiquetas se definió como un VarChar en MySQL en lugar de un tipo JSON.
 5. ¿Por qué se definió como VarChar? Por una configuración inicial rápida en las migraciones de Prisma sin prever la necesidad de guardar arreglos de datos.
- **Causa raíz:** Incompatibilidad en la serialización de datos (Array vs String) entre el

frontend y el modelo de datos de Prisma.

- **Consecuencia:** Bloqueo en la personalización del perfil del trabajador, afectando la experiencia del cliente final al buscar servicios específicos.
- **Acción propuesta:** Implementar un Pipe de transformación en NestJS que convierta el Array a formato JSON string antes de guardarlo, y lo parsee de vuelta al enviarlo al frontend.

3. Análisis del BUG-005: Bloqueo de guardado por biografía vacía

- **Problema presentado:** El sistema no permite actualizar el portafolio si el campo de biografía está vacío.
- **Técnica de los 5 ¿Por qué?:**
 1. ¿Por qué no permite actualizar? Porque el servidor responde con un error HTTP 400 (Bad Request).
 2. ¿Por qué arroja un error 400? Porque la validación de los datos de entrada en el backend falla.
 3. ¿Por qué falla la validación? Porque el campo de biografía está siendo tratado como un dato estrictamente obligatorio.
 4. ¿Por qué es estrictamente obligatorio? Porque al construir la clase DTO (Data Transfer Object) en NestJS, no se especificó que este campo podía omitirse.
 5. ¿Por qué no se especificó? Por la ausencia del decorador de validación adecuado durante el desarrollo del módulo.
- **Causa raíz:** Ausencia del decorador `@IsOptional()` en el DTO de actualización del portafolio dentro de la arquitectura de NestJS, lo que fuerza a que el campo nunca pueda estar vacío.
- **Consecuencia:** Obliga al usuario a ingresar datos de relleno, generando frustración y una mala experiencia de usabilidad.
- **Acción propuesta:** Agregar el decorador `@IsOptional()` en el DTO de NestJS correspondiente y configurar el componente de React para permitir envíos con el campo vacío.